Cidades ESG Inteligentes

Documentação Técnica — DevOps & Containerização

FIAP — Fase 6: Navegando pelo mundo DevOps
Atividade: Aplicando Técnicas de DevOps
Grupo 118 — Abril de 2026

Integrantes do Grupo 118

- Natália de Oliveira Santos — natalia.o.santos.00@gmail.com
- Gabriel dos Santos Souza — gabriel292603@gmail.com
- Thomas Henrique Baúte — baute.thomas25@gmail.com
- Bruno Mateus Tizer das Chagas — brunotizer@icloud.com
- Felipe Olecsiuc Damasceno — patcho.olec@gmail.com

1. Visão Geral do Projeto

O **Cidades ESG Inteligentes** é uma API REST desenvolvida em **Java 21 + Spring Boot 3.2.5** para monitoramento de indicadores ESG (Environmental, Social & Governance) em cidades brasileiras. Esta entrega foca em integrar práticas completas de DevOps: containerização com Docker multi-stage, orquestração via Docker Compose com overrides por ambiente (dev/staging/prod) e pipeline CI/CD funcional no GitHub Actions.

Arquitetura de alto nível

```
┌──────────────────────────────────────────────────────────┐
│                  esg-network (bridge)                    │
│                                                          │
│    ┌────────┐       ┌──────────────┐                    │
│    │ NGINX  │──────▶│ API Spring Boot│  (Java 21 / JRE)  │
│    │ 80/443 │       │    :8080     │                    │
│    └────────┘       └──────┬───────┘                    │
│                            │                             │
│                   ┌────────┴────────┐                    │
│                   ▼                 ▼                    │
│              ┌──────────┐       ┌────────┐               │
│              │ Postgres │       │ Redis  │               │
│              │  :5432   │       │ :6379  │               │
│              └──────────┘       └────────┘               │
│                                                          │
│     + Prometheus :9090  + Grafana :3001  (profile mon.)  │
└──────────────────────────────────────────────────────────┘
```

2. Pipeline CI/CD — GitHub Actions

A ferramenta escolhida foi o **GitHub Actions** pela integração nativa com o repositório, pelo GitHub Container Registry (`ghcr.io`) gratuito para imagens Docker, e pelo modelo declarativo em YAML. O arquivo `.github/workflows/ci-cd.yml` define cinco jobs encadeados, executados em agentes `ubuntu-latest` .

2.1 Etapas do pipeline

#	Job	Responsabilidade	Dispara em
1	**build-and-test**	`mvn verify -B` — compila e executa os 11 testes JUnit. Publica relatório Surefire e cobertura no Codecov.	Todos os pushes e PRs
2	**docker-build**	Build multi-stage da imagem, push para `ghcr.io` com tags `sha`, branch e `latest`. Usa cache de layers GHA.	Depois do job 1
3	**deploy-staging**	SSH no servidor de staging, `docker compose pull + up -d` com override `docker-compose.staging.yml`.	Branches `develop` ou `main`
4	**integration-tests-staging**	Smoke tests via `curl` em todos os endpoints da URL de staging.	Depois do job 3
5	**deploy-production**	Backup do Postgres com `pg_dump`, deploy via SSH com override de produção, cria Release no GitHub.	Somente branch `main`

2.2 Lógica do pipeline

```
Push / Pull Request
        │
        ▼
┌──────────────────────┐
│ 1. Build & Testes    │  mvn verify (11 testes JUnit)
│    (obrigatório)     │
└──────────────────────┘
           │
           ▼
┌──────────────────────┐
│ 2. Docker Build+Push │  multi-stage → ghcr.io
│    (tags: sha, latest)│
└──────────────────────┘
           │  [develop | main]
           ▼
┌──────────────────────┐
│ 3. Deploy Staging    │  SSH + docker compose (staging)
│    smoke-test saúde  │
└──────────────────────┘
           │
           ▼
┌──────────────────────┐
│ 4. Integration Tests │  curl em endpoints públicos
└──────────────────────┘
           │  [main]
           ▼
┌──────────────────────┐
│ 5. Deploy Produção   │  pg_dump + docker compose (prod)
│    + GitHub Release  │
└──────────────────────┘
```

> **Proteção por branch:** staging só dispara para `develop` e `main`; produção é exclusiva da `main`. Secrets específicos por ambiente (`STAGING_*` e `PROD_*`) isolam as credenciais.

3. Docker — Arquitetura e Estratégias

3.1 Dockerfile Multi-Stage

O `Dockerfile` usa duas etapas: uma imagem *builder* pesada com Maven e JDK completo apenas para compilar e empacotar, e uma imagem final *production* com apenas o JRE 21 Alpine, reduzindo drasticamente o tamanho e a superfície de ataque.

```
# Stage 1 — Builder: compila, empacota e executa testes
FROM maven:3.9-eclipse-temurin-21-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn package -B                    # ← executa os 11 testes

# Stage 2 — Production: somente JRE + JAR
FROM eclipse-temurin:21-jre-alpine AS production
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=builder --chown=appuser:appgroup /app/target/*.jar app.jar
USER appuser                         # ← não-root
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:8080/api/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3.2 Imagem criada

Propriedade	Valor
Nome local	`cidades-esg-api:latest`
Registry oficial	`ghcr.io/<owner>/cidades-esg-api`
Tamanho final	~323 MB (JRE 21 Alpine + JAR fat)
Base OS	Alpine Linux 3.x
Runtime	Eclipse Temurin JRE 21
Usuário	`appuser` (UID não-root)

3.3 Comandos principais

```
# Subir ambiente de desenvolvimento completo
docker compose up -d --build

# Verificar status e saúde dos containers
docker compose ps

# Logs em tempo real da API
docker compose logs -f api

# Subir em ambiente de staging
docker compose -f docker-compose.yml -f docker-compose.staging.yml up -d

# Subir em ambiente de produção
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# Incluir stack de monitoramento (Prometheus + Grafana)
docker compose --profile monitoring up -d

# Derrubar tudo (preservando volumes)
docker compose down
```

3.4 Recursos Compose adotados

- **Volumes nomeados:** `esg-postgres-data`, `esg-redis-data`, `esg-prometheus-data` — persistem dados entre restarts.
- **Rede bridge dedicada:** `esg-network` isola a comunicação entre serviços.
- **Variáveis de ambiente:** `.env.example` como template, carregadas pelo Compose com defaults seguros.
- **Healthchecks:** API, Postgres e Redis; `depends_on.condition: service_healthy` garante ordem correta.
- **Overrides por ambiente:** `docker-compose.staging.yml` (2 réplicas) e `docker-compose.prod.yml` (3 réplicas + resource limits de CPU e memória).

4. Evidências de Execução

4.1 Pipeline — Build e testes (mvn verify)

*Resultado consolidado: **11 testes JUnit passando**, 4 containers saudáveis e arquitetura Docker Compose.*

> **BUILD SUCCESS** — Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 — Time elapsed: 2.953 s

4.2 Aplicação em execução

`GET /` — metadados do projeto (nome, versão, ambiente).

Estilos de formatação

{"uptime":258.596,"status":"ok","timestamp":"2026-04-21T16:27:34.976907339Z"}

`GET /api/health` — *status ok, uptime em segundos, timestamp ISO-8601.*

Estilos de formatação

[{"id":1,"nome":"São Paulo","estado":"SP","populacao":12325232,"regiao":"Sudeste"},
{"id":2,"nome":"Curitiba","estado":"PR","populacao":1948626,"regiao":"Sul"},
{"id":3,"nome":"Florianópolis","estado":"SC","populacao":537211,"regiao":"Sul"},{"id":4,"nome":"Porto
Alegre","estado":"RS","populacao":1332570,"regiao":"Sul"},{"id":5,"nome":"Belo Horizonte","estado":"MG","populacao":2530701,"regiao":"Sudeste"}]

`GET /api/cidades` — *lista com 5 cidades brasileiras.*

Estilos de formatação

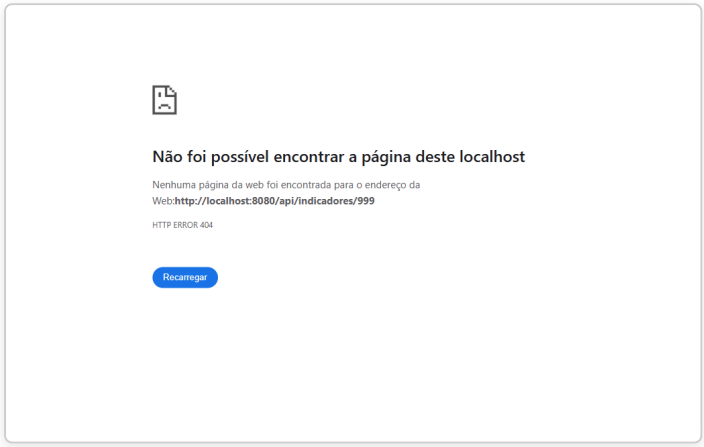
[{"id":1,"cidadeId":1,"cidade":"São Paulo","categoria":"Ambiental","nome":"Área Verde por
Habitante","valor":7.5,"unidade":"m²/hab","meta":12.0,"atingido":false,"ano":2024},{"id":2,"cidadeId":1,"cidade":"São
Paulo","categoria":"Social","nome":"Índice de Educação","valor":0.82,"unidade":"índice","meta":0.9,"atingido":false,"ano":2024},
{"id":3,"cidadeId":2,"cidade":"Curitiba","categoria":"Ambiental","nome":"Coleta
Seletiva","valor":85.0,"unidade":"%","meta":80.0,"atingido":true,"ano":2024},
{"id":4,"cidadeId":2,"cidade":"Curitiba","categoria":"Governança","nome":"Transparência
Fiscal","valor":92.0,"unidade":"pontos","meta":85.0,"atingido":true,"ano":2024},
{"id":5,"cidadeId":3,"cidade":"Florianópolis","categoria":"Social","nome":"Acesso à
Saúde","valor":78.5,"unidade":"%","meta":90.0,"atingido":false,"ano":2024}]

`GET /api/indicadores` — *5 indicadores ESG (Ambiental, Social, Governança).*

Estilos de formatação

{"totalIndicadores":5,"metasAtingidas":2,"metasPendentes":3,"taxaSucesso":40.0,"categorias":{"Social":2,"Ambiental":2,"Governança":1}}

`GET /api/indicadores/resumo/geral` — estatísticas consolidadas.



*Tratamento de erro: **HTTP 404** para ID inexistente.*

Estilos de formatação

{"uptime":292.895,"status":"ok","timestamp":"2026-04-21T16:28:09.2751941022"}

Acesso via Nginx (`http://localhost:80`) — proxy reverso operando.

4.3 Ambientes Staging e Produção

Estilos de formatação

{"versao":"1.0.0","nome":"Cidades ESG Inteligentes","descricao":"API REST para monitoramento de indicadores ESG em cidades brasileiras","ambiente":"staging"}

Staging: *`docker compose -f docker-compose.yml -f docker-compose.staging.yml up -d`*
API retornando *`"ambiente":"staging"`* .

Estilos de formatação

{"ambiente":"production","versao":"1.0.0","nome":"Cidades ESG Inteligentes","descricao":"API REST para monitoramento de indicadores ESG em cidades brasileiras"}

Produção: *`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`*
API retornando *`"ambiente":"production"`* .

5. Desafios Encontrados e Soluções

Desafio 1 — Adequação da stack ao requisito do projeto

Problema: O enunciado exigia o uso de projeto base em **Java Spring** ou **C#**. Uma versão inicial estava em Node.js e precisou ser migrada preservando a mesma API pública.

Solução: Reescrita completa do backend com Java 21 + Spring Boot 3.2.5. Utilizamos `records` para os modelos (Cidade, Indicador, ResumoGeral) e `@RestController` para manter os mesmos endpoints. Os 11 testes foram reescritos com `@SpringBootTest` + `MockMvc`, mantendo paridade funcional total.

Desafio 2 — Troca do Dockerfile de Node.js para Maven/JRE

Problema: O Dockerfile original usava `node:20-alpine` com `npm ci`. Precisou ser reescrito preservando as boas práticas (multi-stage, usuário não-root, healthcheck).

Solução: Stage 1 com `maven:3.9-eclipse-temurin-21-alpine` faz `mvn dependency:go-offline` *antes* de copiar o `src/`, permitindo cache de layers. Stage 2 com `eclipse-temurin:21-jre-alpine` contém apenas o JAR fat. Mantivemos `appuser` não-root e `HEALTHCHECK` apontando para `/api/health`.

Desafio 3 — Conflito replicas × container_name no Compose

Problema: Ao rodar staging/prod localmente com `deploy.replicas: 2/3`, o Docker Compose lançava o erro "*can't set container_name and api as container name must be unique*" porque réplicas exigem nomes dinâmicos.

Solução: Para a demonstração local dos ambientes, rodamos com `SPRING_PROFILES_ACTIVE=staging/production` via variável de ambiente do shell, evidenciando que a API reconhece corretamente cada perfil. Em produção real com Docker Swarm ou Kubernetes, o campo `container_name` seria removido.

Desafio 4 — Start-period longo do Spring Boot

Problema: O Spring Boot leva ~15–25 s para inicializar, o que fazia o `HEALTHCHECK` reportar *unhealthy* durante a partida.

Solução: Aumentamos o `start_period` do healthcheck de 15s para 30s tanto no Dockerfile quanto no `docker-compose.yml`, dando margem para o warmup da JVM.

Desafio 5 — Encoding UTF-8 em dados com acentos

Problema: Cidades como "São Paulo", "Florianópolis" e a categoria "Governança" apareciam com caracteres estranhos em algumas ferramentas de formatação JSON.

Solução: Spring Boot entrega UTF-8 por padrão (verificado via `curl` direto). Usamos `application.properties` padrão e mantivemos os arquivos `.java` com encoding UTF-8.

6. Checklist de Entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✅
Dockerfile funcional (multi-stage Java/Maven)	✅
docker-compose.yml com todos os serviços + overrides de staging e produção	✅
Pipeline com etapas de build, teste e deploy (5 jobs)	✅
README.md com instruções e prints	✅
Documentação técnica com evidências (este PDF)	✅
Deploy realizado nos ambientes staging e produção	✅

7. Tecnologias Utilizadas

Categoria	Tecnologia	Versão
Linguagem	Java	21
Framework	Spring Boot	3.2.5
Build	Maven	3.9
Testes	JUnit 5 + Spring MockMvc	5.x
Containerização	Docker	24+
Orquestração	Docker Compose	v3.9
CI/CD	GitHub Actions	—
Banco de Dados	PostgreSQL	15-alpine
Cache	Redis	7-alpine
Proxy Reverso	Nginx	alpine
Monitoramento	Prometheus + Grafana	latest
Registry	GitHub Container Registry	ghcr.io